NETWORK DESIGN · GCP · EGRESS

Agent Gateway Egress Design

How every Agent Runtime (Agent Engine) agent reaches internet APIs and internal services through Google's Agent Gateway and your existing Apigee X platform. Nothing new is deployed in the hubs and no new VMs are run. The design keeps your pattern: Zscaler for internet egress, no VPC peering, NAT only on Zscaler subnets, private DNS only, prod and nonprod apart, east and west in every layer. Ingress is covered separately.

Draft v4 for review · 22 Oct 2026 · scope: GCP egress only

Decisions recorded so far

- Agent Gateway is the org's chosen control point. Zscaler does all content inspection.
- Every hub has Zscaler Cloud Connectors in both regions, separately for prod and nonprod. Zscaler will never get a NIC in a new VPC.
- All DNS stays private: Infoblox is the enterprise DNS, and nothing may resolve publicly. Every CIDR is reserved in IPAM.
- **New in v4:** no new VM stack. The existing Apigee X platform replaces the v3 egress proxy MIG. Apigee runs in its own service project wired to one hub, is published with a PSC service attachment, and already serves consumers in every hub. Agent Gateway stays in separate projects.

The design in one page

Agent Gateway is the agent's only exit

Each agent is bound to a Google-managed egress gateway, and all its outbound traffic goes through it. The gateway checks the agent's identity and allowlist. Agents no longer get their own PSC interface.

Four egress cells, no VMs

One cell per environment and region: prod-east, prod-west, nonprod-east, nonprod-west. A cell is one gateway plus a small VPC holding only PSC endpoints and private DNS. It has no default route, no NAT and no VMs.

Apigee is the way out

The cell's DNS points the agent API hostname at a PSC endpoint for your existing Apigee service attachment. Apigee runs one API proxy per approved destination, adds credentials and quotas, and calls the target over its existing egress, which must go through Zscaler.

Internal APIs go through Apigee too

Apigee already reaches backends in every hub, so internal APIs need no new PSC work. Services that aren't behind Apigee, such as some MCP servers, can still get a direct PSC endpoint as an exception.

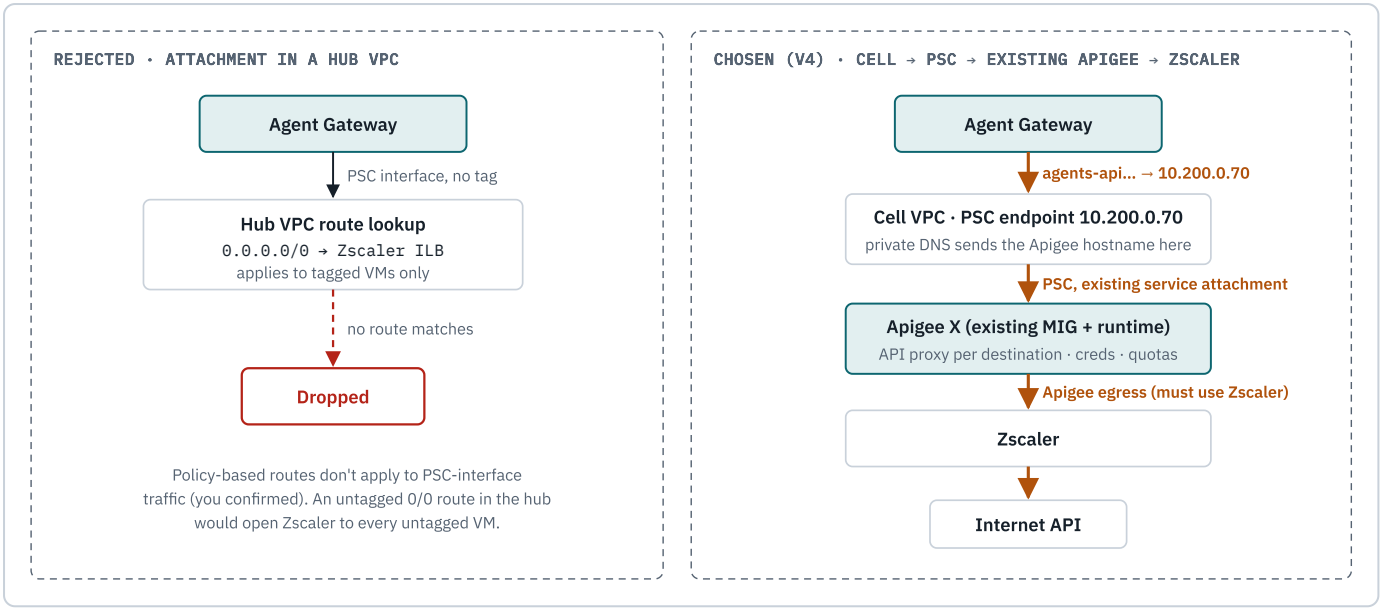
Design rules

1. All agent egress goes through Agent Gateway. No agent keeps a direct PSC interface.
2. Agents reach the internet only through Apigee API proxies. Arbitrary websites and downloads are not reachable.
3. Apigee's calls to the internet go through Zscaler. This must be confirmed before go-live (see the warning below).
4. Prod and nonprod never share a gateway, a cell VPC or an Apigee environment.
5. East and west are built identically, and each region works on its own.
6. Cell VPCs have no default route, no NAT and no VMs. PSC endpoints are the only way out.
7. No public DNS anywhere. The gateway resolves only through the cell's private zones.
8. Every range, including the isolated cell VPCs, is reserved in IPAM before it's created.

How traffic reaches the internet

The gateway reaches your network through a PSC interface, which can't carry a network tag. In a hub VPC the only internet route needs the Zscaler tag, so the gateway's traffic would be dropped there. That's why each cell has its own VPC: it gives the gateway a private DNS view and PSC endpoints without touching any hub.

From the cell, the gateway sends agent API calls to a PSC endpoint for Apigee. Apigee is already published to every hub through a service attachment, and already calls internet APIs for other consumers. It becomes the agents' exit too. Nothing new runs in a hub.



The gateway never routes anything to the internet. It calls Apigee on a private PSC address, and Apigee makes the internet call.

VERSION	INTERNET EXIT	WHY IT CHANGED
v1	Two-NIC bridge VM into Hub A	Needed IP forwarding and a VM straddling two networks
v2–v3	Envoy egress proxy MIG in Hub A, over PSC	A new VM stack to scale, patch and run
v4	**Existing Apigee X, over its existing PSC service attachment**	**Chosen:** nothing new in the hubs, no VMs, and API-level control (credentials, quotas, logging). Only modelled APIs are reachable

What changes with Apigee

Apigee is an API gateway, not a network exit. That shapes how agents use it.

- **Agents call Apigee's URL, not the real hostname.** An agent calls `https://agents-api.prod.corp.internal/salesforce/…`, not `api.salesforce.com`. SDKs that hard-code a hostname need their base URL overridden.
- **Every destination is an API proxy.** Each internet or internal API an agent needs is modelled once in Apigee. Anything not modelled is unreachable, which is a stronger allowlist than hostnames alone. Package downloads and general web browsing aren't supported.
- **Apigee holds the credentials.** Salesforce, ServiceNow and similar secrets live in Apigee, not in the agent. Each agent or BU authenticates to Apigee with its own key or token and gets its own API product and quota.
- **Two layers of control.** Agent Gateway decides which agent may call which Apigee hostname. Apigee decides which API and operation, how often, and with which

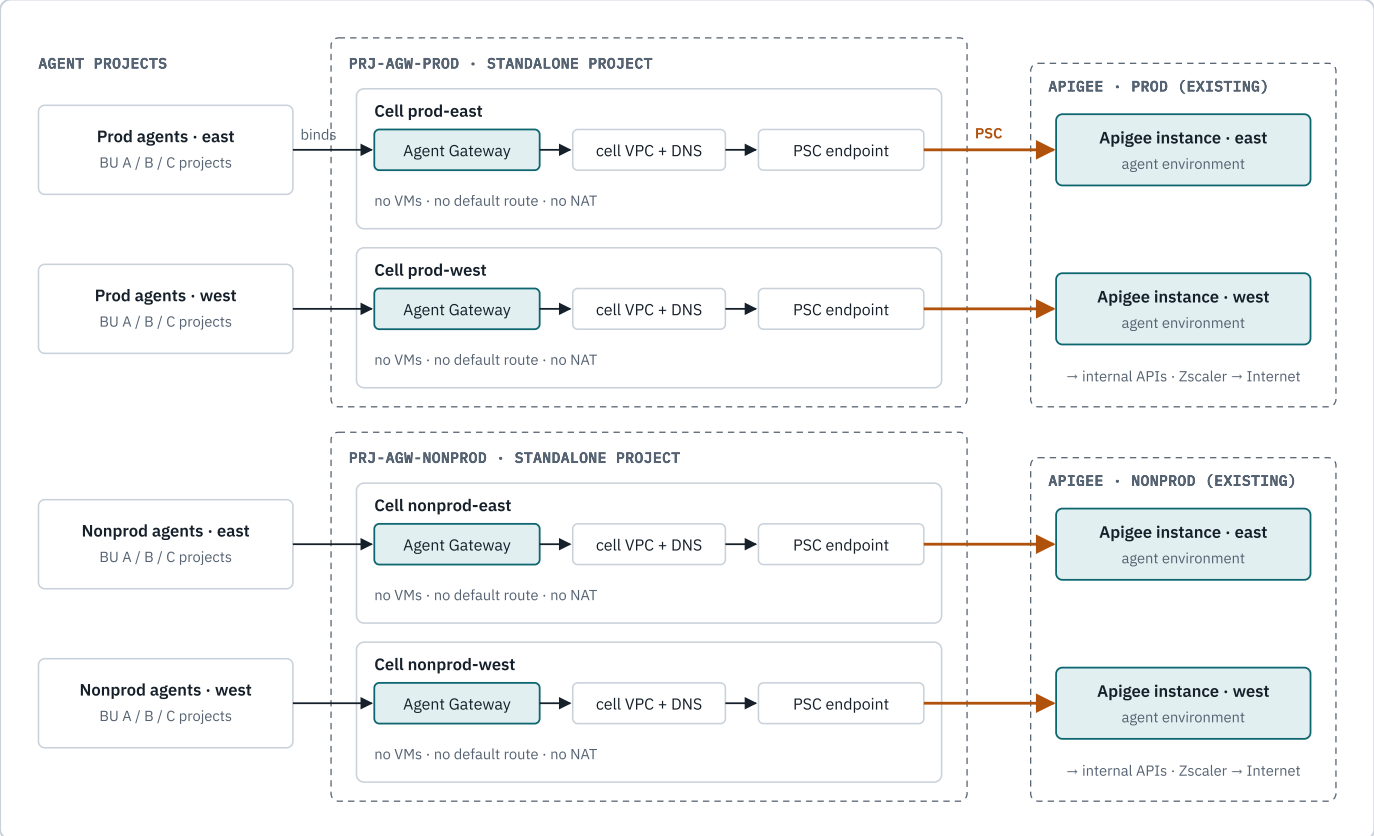
credentials. Optionally, give each external API its own hostname on the agent environment group (for example `salesforce.agents-api.prod.corp.internal`). The gateway's allowlist then also works per API.

- **Streaming and limits.** MCP (streamable HTTP or SSE) and long LLM calls need Apigee streaming turned on and target timeouts raised. Check payload limits for large responses.

> **Confirm before building: how does Apigee reach the internet today?**
>
> The Apigee MIG carries traffic *into* Apigee. Apigee's own calls *out* to internet targets are made by the Apigee runtime, which by default leaves through Apigee-managed Cloud NAT and bypasses Zscaler. If that's how it works today, set Apigee's forward proxy on the agent environment to a Zscaler explicit-proxy endpoint reachable from Apigee. Without that step this design breaks the Zscaler rule.

The four egress cells

A cell is the unit you build, test and run. Agent Gateway is regional, and an agent can only bind to a gateway in its own region. Every agent project binds to the cell matching its environment and region, whichever business-unit hub it belongs to. Each cell consumes the Apigee instance for the same environment and region.

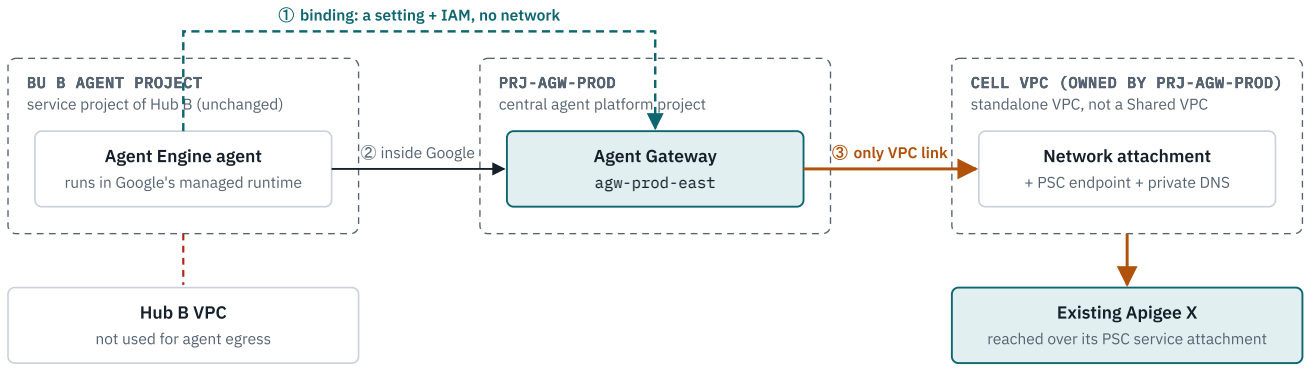


Four cells serve all three business units. Each consumes the existing Apigee service attachment for its environment and region. If Apigee isn't deployed in both regions or both environments today, that gap must be closed first (question 2).

CELL	PROJECT	AGENTS THAT BIND TO IT	CONSUMES OVER PSC
prod-east	`prj-agw-prod`	Prod agent projects in east, all BUs	Prod Apigee, east instance (agent environment)
prod-west	`prj-agw-prod`	Prod agent projects in west, all BUs	Prod Apigee, west instance (agent environment)
nonprod-east	`prj-agw-nonprod`	Nonprod agent projects in east, all BUs	Nonprod Apigee, east instance (agent environment)
nonprod-west	`prj-agw-nonprod`	Nonprod agent projects in west, all BUs	Nonprod Apigee, west instance (agent environment)

How agents connect

Agent projects don't attach to the cell VPC, and they don't need a second host project. Agent Engine doesn't run in any of your VPCs; it runs in Google's managed runtime. The link from an agent to the gateway is a setting on the agent plus an IAM grant, and the traffic between them stays inside Google. The only connection into a VPC belongs to the gateway, and it lands in the cell VPC that `prj-agw-prod` owns.



The agent project keeps its single Shared VPC attachment to its own hub. Only the gateway touches the cell VPC.

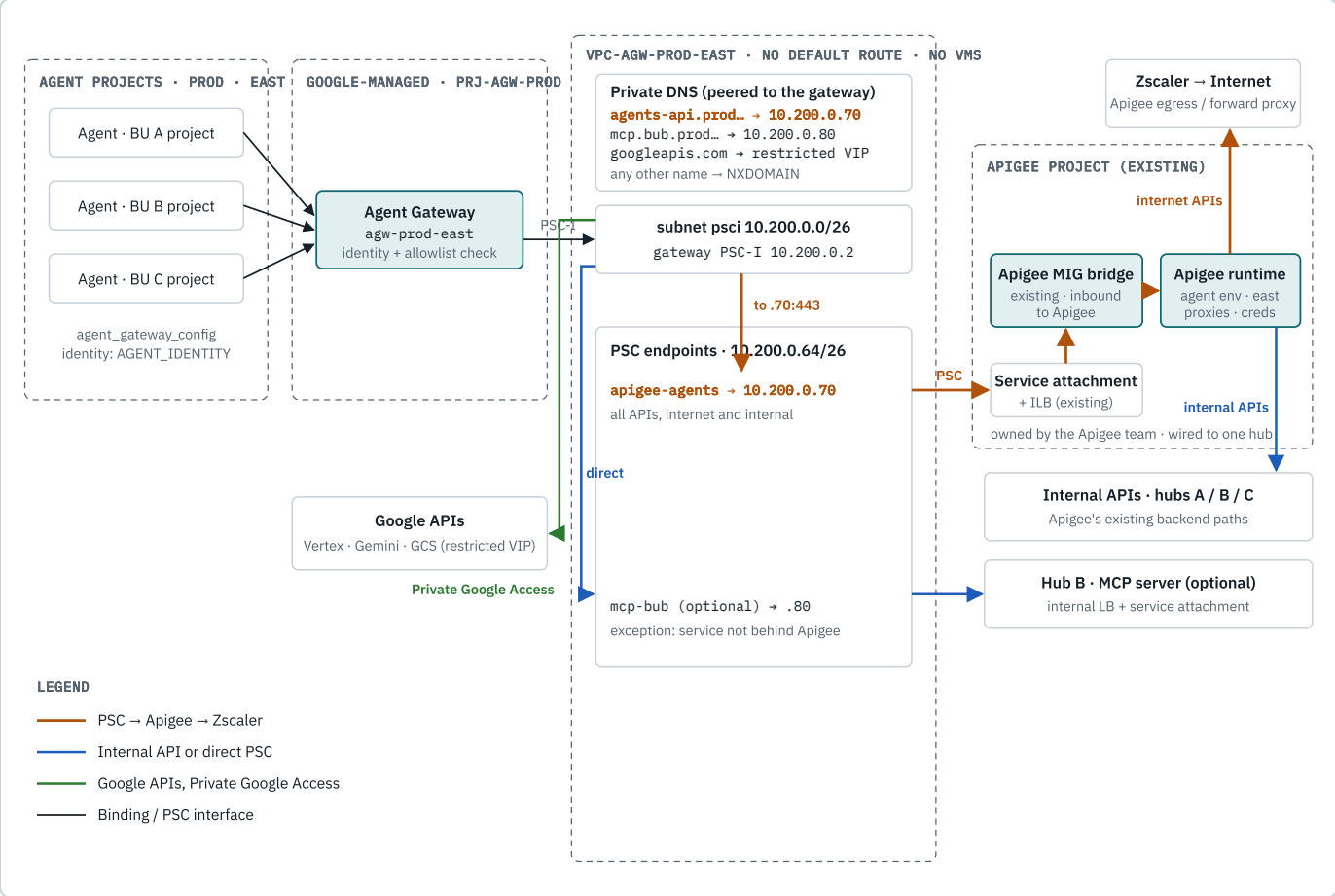
PROJECT	SHARED VPC HOST	CONTAINS	VPC USED FOR AGENT EGRESS
BU agent project (e.g. BU B prod)	Its own hub, unchanged	Agent Engine agents	None. Its old PSC interface is removed
`prj-agw-prod`	None needed	Agent Gateway, Agent Registry, cell VPCs	Its own cell VPCs (east and west)

PROJECT	SHARED VPC HOST	CONTAINS	VPC USED FOR AGENT EGRESS
Apigee project (existing)	One hub, unchanged	Apigee MIG bridge, service attachments	Unchanged; it gains new PSC consumers
Hub host projects	—	Zscaler, backends Apigee already reaches	Unchanged

What a BU agent project needs, all of it settings and IAM: the `agent_gateway_config` binding with `AGENT_IDENTITY`; a small custom role on `prj-agw-prod` for its Agent Engine service agent; registration in Agent Registry; `roles/iap.egressor` grants; and its Apigee credential. Google documents that an egress gateway can be in a different project from the agent, as long as both are in the same region.

Inside one cell

This is the prod-east cell; the other three are identical except for names and ranges. The cell VPC holds one network attachment, one PSC endpoint for Apigee (plus optional direct endpoints), a single route for Google APIs and private DNS. Everything to the right of the PSC endpoint already exists.



One PSC endpoint to Apigee carries almost all agent traffic. Only what's to the left of Apigee is new.

Private DNS

Without a default route, DNS decides where every connection goes. Your rule is that nothing resolves publicly, and Agent Gateway uses public recursive DNS unless told otherwise. So the connectivity template peers the root domain `.` to the cell VPC. Every lookup the gateway makes is answered by the cell's private Cloud DNS zones, which only that VPC can see. With Apigee in front, the zones are tiny.

ZONE IN THE CELL VPC	TYPE	ANSWERS
`.` (root)	Private, authoritative, empty	NXDOMAIN for every name not in a more specific zone. No query ever goes to public DNS.
`googleapis.com.`	Private, authoritative	`*.googleapis.com` → restricted VIP `199.36.153.4/30`
`corp.internal.` (your internal suffixes)	Private, authoritative	The agent Apigee hostname(s) → `10.200.0.70`; any direct-PSC exceptions → their endpoint IPs

- **Closed on purpose.** Google warns that an authoritative root zone returns NXDOMAIN for anything not defined. Here that's the goal. Any Google hostname outside `googleapis.com` needed by platform calls shows up as an NXDOMAIN in the nonprod dry run, and gets a record.
- **Internet hostnames never appear in the cell.** Agents only resolve Apigee's name. Apigee resolves real targets through its own existing path.
- **Infoblox stays the enterprise DNS.** The cell zones are a small private view only the gateway uses. They contain PSC addresses that mean nothing elsewhere. The onboarding pipeline writes them, or Infoblox manages them if your edition integrates with Cloud DNS.
- **The template rarely changes.** Only `.` is peered. Adding an API means adding an Apigee proxy, and a DNS record only if it gets its own hostname.

`verify in lab` Prove in nonprod:

- Cloud DNS accepts a private root zone, and Agent Gateway honours `.` peering to it.
- Whether a bound agent's runtime makes any DNS lookups of its own outside the gateway. Ask Google if logs don't show it.

Traffic flows, step by step

Each flow is written as legs: where the request is, which rule it hits, and what decides the next hop. The addresses come from the prod-east example.

Agent → internet SaaS API

LEG 1 · INSIDE GOOGLE

1. The agent calls `https://agents-api.prod.corp.internal/salesforce/…` with its Apigee credential. The binding sends the call to `agw-prod-east`.
2. The gateway checks the agent's identity and its `iap.egressor` grant on the Apigee hostname.
3. The cell's private `corp.internal.` zone resolves the hostname to `10.200.0.70`.

LEG 2 · CELL VPC → APIGEE OVER PSC

4. The gateway connects from `10.200.0.2` to `10.200.0.70:443`. It validates Apigee's certificate against the corporate CA in its `trustConfig`.
5. The PSC endpoint forwards to Apigee's existing service attachment. The MIG bridge passes it to the Apigee runtime (east).

LEG 3 · APIGEE → ZSCALER → INTERNET

6. The `/salesforce` API proxy checks the agent's credential, applies its quota and swaps in the Salesforce credential.
7. Apigee calls `api.salesforce.com` over its egress path, which goes through Zscaler (via the forward proxy if needed).
8. Zscaler inspects and forwards. The response streams back the same way.

Agent → internal API in any hub

1. The agent calls `https://agents-api.prod.corp.internal/orders/…`.
2. Legs 1 and 2 are the same as the SaaS flow: gateway, cell DNS, PSC endpoint, Apigee.
3. The `/orders` API proxy calls the backend in Hub B over Apigee's existing backend path. That path already serves your other consumers.
4. No Zscaler, no new PSC work, no Interconnect hairpin.

Agent → MCP server not behind Apigee

1. Exception path. The BU publishes the MCP server with a service attachment, and the cell gets a direct PSC endpoint (`10.200.0.80`) and a DNS record.
2. The gateway checks identity and allowlist, then connects straight to the endpoint.
3. Use it only where Apigee can't front the service. Each exception costs a PSC NAT range in the BU hub.

Agent → Gemini / Google APIs

1. The agent calls `{region}-aiplatform.googleapis.com`. Platform calls go through the gateway too, so each Google API hostname must be allowlisted (the agent needs `iap.resources.egressViaIAP`).
2. The cell's `googleapis.com.` zone answers with the restricted VIP.
3. The `199.36.153.4/30` route plus Private Google Access takes it straight to Google, without passing through Apigee or Zscaler.

Agent → anything not approved

1. Calling a real internet hostname directly: NXDOMAIN in the cell, so there's no connection. Even a hard-coded public IP has no route out.
2. Apigee hostname without a grant: denied by the gateway.
3. An Apigee path with no API proxy, or a product the agent isn't subscribed to: rejected by Apigee.

Resource and IP plan

Names and ranges are examples to replace with IPAM reservations. Every cell range must be RFC 1918, as Agent Gateway requires for the attachment subnet. Hubs never see cell ranges because PSC translates addresses, so they can't break routing. Reserve them as unique blocks marked "isolated, not routed" anyway. Nothing new is needed in any hub, except a PSC NAT range for each direct-PSC exception.

CELL	CELL VPC	PSCI (ATTACHMENT)	PSC ENDPOINTS	APIGEE ENDPOINT	CONSUMES
prod-east	`vpc-agw-prod-east`	10.200.0.0/26	10.200.0.64/26	10.200.0.70	Prod Apigee east service attachment

CELL	CELL VPC	PSCI (ATTACHMENT)	PSC ENDPOINTS	APIGEE ENDPOINT	CONSUMES
prod-west	`vpc-agw-prod-west`	10.200.1.0/26	10.200.1.64/26	10.200.1.70	Prod Apigee west service attachment
nonprod-east	`vpc-agw-nonprod-east`	10.201.0.0/26	10.201.0.64/26	10.201.0.70	Nonprod Apigee east service attachment
nonprod-west	`vpc-agw-nonprod-west`	10.201.1.0/26	10.201.1.64/26	10.201.1.70	Nonprod Apigee west service attachment

WHERE	RESOURCE	KEY SETTINGS
Cell	Custom-mode VPC, 2 subnets	Private Google Access on `psci`; default internet route deleted; no Cloud Router, no NAT, no VMs
Cell	1 static route	`199.36.153.4/30` → default internet gateway (Google APIs only)
Cell	Private DNS zones	Closed root `.`, `googleapis.com.`, `corp.internal.` (Apigee hostname + exceptions)
Cell	PSC endpoints	Apigee service attachment for the cell's environment and region; direct-PSC exceptions only when needed
Cell	Agent Gateway + connectivity template	`AGENT_TO_ANYWHERE`, `CENTRALIZED`, `vpcEgress: ALL_TRAFFIC`, DNS peering `.`, corporate CA (signs the Apigee hostname) in `trustConfig`
Cell	Agent Registry + IAP authz	Global + regional registry in the gateway project; entries = Apigee hostname(s) + Google API hostnames; start in `DRY_RUN`
Apigee (existing)	Agent environment + environment group	Per Apigee org (prod, nonprod), attached to the east and west instances; hostname `agents-api.{env}.corp.internal` (plus optional per-API hostnames)
Apigee (existing)	Service attachment accept list	Add `prj-agw-prod` or `prj-agw-nonprod`; check NAT capacity for 2 more endpoints per org
Apigee (existing)	Egress to internet	Must go through Zscaler: existing path, or forward proxy on the agent environment → Zscaler explicit endpoint
Apigee (existing)	API products, developer apps	One app per agent (or per BU) with its own credential and quota

Implementation steps

Build in this order: nonprod-east, nonprod-west, prod-east, prod-west. Phase 3 repeats per cell, so write it as one Terraform module instantiated four times. The commands show prod-east, with `EAST` standing for your east region. `documented` marks steps taken from Google's docs. `verify in lab` marks steps whose exact syntax or behaviour you should prove in nonprod first.

PHASE 0 Decisions and approvals

Apigee · Network · Security · Zscaler · DNS teams

- **Apigee team:** agree to host an agent environment per org, accept the agent projects on the service attachments, and confirm MIG and runtime capacity for agent traffic. Confirm instances in both regions.
- **Zscaler team:** confirm how Apigee's internet calls reach Zscaler today. If they don't, provide an explicit-proxy endpoint Apigee can reach, for the agent environment's forward proxy.
- **Security:** approve the four cell VPCs (PSC-only, no VMs) and "agents reach only modelled APIs".
- **IPAM:** reserve per cell a psci /26 and a PSC endpoint /26 (one /25 per cell, marked isolated).
- **DNS / Infoblox:** agree the cell zones are a separate private view, and who writes them.
- **Org policy:** `constraints/iam.managed.disableAccessPolicyBindings` off in the agw projects, so Agent Gateway's IAM access policies work. `documented` No IP-forwarding, multi-NIC or Shared VPC exceptions are needed.

PHASE 1 Projects

Platform

- Create `prj-agw-prod` and `prj-agw-nonprod` in an agent-platform folder. They don't need to be Shared VPC service projects, because nothing in them uses a hub subnet.
- Enable APIs: Compute, Cloud DNS, Network Services, Network Security, Certificate Manager, IAP, Agent Registry, Vertex AI.

PHASE 2 Apigee agent environment (per Apigee org)

Apigee team

1. Create an environment, for example `agents`, and attach it to the east and west instances.
2. Create an environment group with hostname `agents-api.prod.corp.internal`, using a certificate from your corporate CA. Optionally add one hostname per external API.
3. If Apigee's internet calls don't already go through Zscaler, set the environment's forward proxy to the Zscaler explicit endpoint. Keep it on this environment only, so other consumers are unaffected. If Zscaler inspects, give the target endpoints a truststore with the Zscaler root CA. `verify in lab`
4. Add `prj-agw-prod` to the consumer accept list of the east and west service attachments.
5. Build a template API proxy (agent credential check → quota → credential injection → target, with streaming on and longer timeouts). Create one proxy per destination from it.

6. Create API products and one developer app per agent or BU. Store each credential in Secret Manager in the agent's project.

PHASE 3 Cell VPC (per cell) Platform

```
# VPC and subnets
gcloud compute networks create vpc-agw-prod-east --project=prj-agw-prod --subnet-mode=custom --bgp-routing-mode=regional
gcloud compute networks subnets create snet-agw-psci-east --project=prj-agw-prod --network=vpc-agw-prod-east --region=EAST --range=10.200.0.0/26 --enable-private-ip-google-access
gcloud compute networks subnets create snet-agw-pscep-east --project=prj-agw-prod --network=vpc-agw-prod-east --region=EAST --range=10.200.0.64/26

# Remove the auto-created internet route; add the Google APIs route only
gcloud compute routes list --project=prj-agw-prod --filter="network~vpc-agw-prod-east AND destRange=0.0.0.0/0"
gcloud compute routes delete DEFAULT_ROUTE_NAME --project=prj-agw-prod
gcloud compute routes create rt-agw-prod-east-restricted-apis --project=prj-agw-prod --network=vpc-agw-prod-east --destination-range=199.36.153.4/30 --next-hop-gateway=default-internet-gateway --priority=100

# PSC endpoint to the existing Apigee service attachment (east)
gcloud compute addresses create ip-psc-apigee-east --project=prj-agw-prod --region=EAST --subnet=snet-agw-pscep-east --addresses=10.200.0.70
gcloud compute forwarding-rules create psc-apigee-east --project=prj-agw-prod --region=EAST --network=vpc-agw-prod-east --address=ip-psc-apigee-east --target-service-attachment=projects/APIGEE_PROJECT/regions/EAST/serviceAttachments/APIGEE_SA_EAST

# Private DNS: closed root, Google APIs, internal names
gcloud dns managed-zones create pz-root-closed --project=prj-agw-prod --dns-name=. --visibility=private --networks=vpc-agw-prod-east --description="closed DNS: nothing resolves publicly"
gcloud dns managed-zones create pz-googleapis --project=prj-agw-prod --dns-name=googleapis.com. --visibility=private --networks=vpc-agw-prod-east --description="restricted VIP"
gcloud dns record-sets create restricted.googleapis.com. --project=prj-agw-prod --zone=pz-googleapis --type=A --ttl=300 --rrdatas=199.36.153.4,199.36.153.5,199.36.153.6,199.36.153.7
gcloud dns record-sets create "*.googleapis.com." --project=prj-agw-prod --zone=pz-googleapis --type=CNAME --ttl=300 --rrdatas=restricted.googleapis.com.
gcloud dns managed-zones create pz-corp-internal --project=prj-agw-prod --dns-name=corp.internal. --visibility=private --networks=vpc-agw-prod-east --description="Apigee + PSC exceptions"
gcloud dns record-sets create agents-api.prod.corp.internal. --project=prj-agw-prod --zone=pz-corp-internal --type=A --ttl=300 --rrdatas=10.200.0.70
```

> **Gate:** attach a temporary test VM (no tag, no external IP) to a spare subnet in the cell VPC. Call a test proxy, for example `curl -s https://agents-api.prod.corp.internal/echo/` , whose target is a public test site. Pass: the call works, Apigee shows the call, and Zscaler logs show Apigee's request. Then delete the VM.

PHASE 4 Agent Gateway (per cell) Platform

1. Load the corporate root CA that signs the Apigee hostname into a regional Certificate Manager `TrustConfig` . The Zscaler root isn't needed here, because the gateway never talks to Zscaler.

documented

2. Create the network attachment on the psci subnet with automatic acceptance. documented
3. Create the connectivity template and the gateway. verify in lab The field names come from the docs; check the nesting and import commands against the current API reference.
4. Create the IAP authorization extension and policy in `DRY_RUN`. documented

```
gcloud certificate-manager trust-configs import tc-corp-ca-east --project=prj-agw-prod --location=EAST
--source=tc-corp-ca.yaml
gcloud compute network-attachments create na-agw-prod-east --project=prj-agw-prod --region=EAST --
subnets=snet-agw-psci-east --connection-preference=ACCEPT_AUTOMATIC
```

```
# connectivity-template.yaml (agent connectivity template)
accessPath: AGENT_TO_ANYWHERE
deploymentModel: CENTRALIZED
egressNetworkConfig:
  networkAttachment: projects/prj-agw-prod/regions/EAST/networkAttachments/na-agw-prod-east
  vpcEgress: ALL_TRAFFIC
  dnsPeeringConfig:
    domains: ["."]               # every lookup goes to the cell's private zones; nothing public
    targetNetwork: projects/prj-agw-prod/global/networks/vpc-agw-prod-east
tlsConfig:
  trustConfig: projects/prj-agw-prod/locations/EAST/trustConfigs/tc-corp-ca-east
  additionalRoots: PUBLICLY_TRUSTED_ROOTS

# agent-gateway.yaml  →  gcloud network-services agent-gateways import agw-prod-east --location=EAST -
-source=agent-gateway.yaml
name: agw-prod-east
googleManaged:
  governedAccessPath: AGENT_TO_ANYWHERE
agentConnectivityTemplate: projects/prj-agw-prod/locations/EAST/agentConnectivityTemplates/act-agw-
prod-east
registries:
  - //agentregistry.googleapis.com/projects/prj-agw-prod/locations/global
  - //agentregistry.googleapis.com/projects/prj-agw-prod/locations/EAST

# authz extension: service iap.googleapis.com, failOpen false, timeout 1s,
#   metadata.iapPolicyVersion "V2", iamEnforcementMode "DRY_RUN"
# authz policy:   policyProfile REQUEST_AUTHZ, action CUSTOM, target = the gateway
```

- Register in Agent Registry: the Apigee hostname(s) and the Google API hostnames agents use, each exact, with no wildcards.
- Leave Model Armor off on egress unless security wants it. Zscaler and Apigee already cover content.
- The cell VPC must not use Cloud NGFW Enterprise or Network Security Integration; Agent Gateway returns 503 on those networks.

PHASE 5 Bind agents (per agent project) BU agent teams · Platform

1. Grant each agent project's Runtime service agent a custom role on the gateway project with `networkservices.agentGateways.get` and `networkservices.operations.get`. documented

2. Change the agent's tools to call Apigee URLs and send its Apigee credential.
3. Deploy or update the agent with the binding and agent identity, and remove `psc_interface_config`.
4. Register the agent in Agent Registry and grant `roles/iap.egressor` on the Apigee hostname(s).

```
remote_agent = client.agent_engines.create(
    agent=app,
    config={
        "identity_type": "AGENT_IDENTITY",
        "agent_gateway_config": {
            "agent_to_anywhere_config": {
                "agent_gateway": "projects/prj-agw-prod/locations/EAST/agentGateways/agw-prod-east"
            }
        },
        "env_vars": {"AGENTS_API_BASE": "https://agents-api.prod.corp.internal"},
        # psc_interface_config removed: all egress now goes through the gateway
    },
)
```

PHASE 6 ### Test, then enforce `Platform · Apigee · Security`

- Run every cell in `DRY_RUN` for at least one business cycle. Read the IAP decisions, the cell's NXDOMAIN logs and Apigee's analytics, and fill the gaps.
- Run the test plan, then switch the authorization extension to enforce.
- Repeat for the next cell in the rollout order.

Moving existing agents

- **Agents created before 29 April 2026** can't bind to a gateway. Redeploy them as new agents.
- **Changing an agent's gateway binding archives all its existing revisions**, and that can't be undone. Bind once, during a planned change, when all traffic is on the latest revision.
- **Tool changes come first.** Each agent's tools must call Apigee URLs before binding, because direct internet hostnames stop resolving once it's bound.
- **Order for each agent:** build its Apigee proxies, change its tools, bind it in nonprod with the gateway in `DRY_RUN`, and confirm nothing is denied. Then remove its PSC interface and repeat in prod.
- **Security Command Center's Agent Engine Threat Detection** is turned off for agents bound to a gateway. Tell the SOC before go-live.

Test plan

TEST	HOW	PASS
Internet via Apigee and Zscaler	Agent calls a SaaS API through its Apigee proxy	Succeeds; Apigee analytics and Zscaler logs both show it; gateway log shows the agent identity
Direct internet hostname	Agent calls `api.salesforce.com` directly	NXDOMAIN; no connection
No public DNS	Check Cloud DNS query logs on the cell VPC	Every answer comes from a private zone; nothing goes to a public resolver
Hard-coded public IP	Test VM in the cell calls a public IP	Dropped: no route
No gateway grant	Agent without `iap.egressor` calls the Apigee hostname	Denied by the gateway
No Apigee subscription	Agent calls an API its app isn't subscribed to	Rejected by Apigee (401/403)
Apigee egress bypass	Watch a target that logs source IPs	The request arrives from Zscaler egress IPs, never from Apigee's own NAT IPs
Streaming	MCP or LLM streaming call through Apigee	Streams without buffering or timeout
Internal API via Apigee	Agent calls an internal API in Hub C	Succeeds; absent from Zscaler
Environment isolation	Nonprod agent calls the prod Apigee hostname	Fails: nonprod cells have no prod records or endpoints, and the prod accept list rejects them
Google APIs	Agent calls Gemini	Succeeds through the restricted VIP; not in Apigee or Zscaler
Region independence	Remove the nonprod east PSC endpoint	East agents fail; west unaffected

Operations and ownership

AREA	OWNER	WHAT THEY RUN
Cells (gateway, cell VPC, DNS, endpoint)	Agent platform team	Cell module ×4, registry and grants, dry-run reviews. **No VMs**

AREA	OWNER	WHAT THEY RUN
Apigee agent environment and proxies	Apigee team (with BU requests)	Environment, API proxies, products, apps, credentials, quotas, capacity of the existing MIG and runtime
Apigee internet egress	Apigee + Zscaler teams	The path from Apigee to Zscaler, and Zscaler policy for Apigee's egress
Direct-PSC exceptions	BU (publish) + platform (consume)	Service attachment, endpoint, DNS record, registry entry

- **Runbook – new external API for agents:** Apigee builds the proxy from the template, stores the target credential and adds it to a product. The BU's app subscribes. No network or DNS change, unless the API gets its own hostname.
- **Runbook – new agent:** binding, gateway role, registry entry, `iap.egressor` grant, Apigee app and credential.
- **Runbook – corporate CA rotation:** update the PEM in all four TrustConfigs before the old CA expires. It isn't automated.
- **Monitor:** PSC endpoint status; gateway 498 and 503 rates; IAP denials; cell NXDOMAIN counts; Apigee latency, errors and quota hits for the agent environment; Zscaler traffic from Apigee.
- **Shared dependency:** Apigee becomes part of every agent's egress. Agree an SLO with the Apigee team, and keep agent quotas in the agent environment so agents can't affect other Apigee customers.

Open questions

1. **How does Apigee reach the internet today?** Through Zscaler (and how), or Apigee's own NAT? If the latter, the agent environment needs a forward proxy to a Zscaler explicit endpoint.
2. **Is Apigee deployed in east and west, for both prod and nonprod?** Each cell needs a same-region service attachment in its environment.
3. **Can the Apigee team host the agent environment and accept the agw projects?** Including capacity on the existing MIG bridge and an SLO.
4. **Are all agent destinations APIs?** Anything else (web browsing, package downloads) isn't reachable in this design.
5. **How do agents authenticate to Apigee?** API key per agent app (simplest), or a Google-signed identity token if agents can mint one.
6. **Which corporate CA signs the Apigee hostname?** Its root goes into the gateway's TrustConfig in each region.

7. **Are VPC Service Controls in place around the agent projects, and is Cloud NGFW Enterprise used anywhere?** VPC-SC needs ALL_TRAFFIC, which we use. NGFW Enterprise breaks Agent Gateway.

Sources

- Agent Gateway overview
- Set up Agent Gateway
- Set up VPC connectivity for Agent Gateway (DNS peering, root-zone guidance)
- Route Agent Runtime traffic through Agent Gateway
- Apigee: provisioning NAT IPs (default internet egress)
- How routing works in Apigee X
- Controlling Apigee X's internet egress with a firewall appliance